LedgerLens — Oracle PL/SQL Transaction Monitoring and Audit System

Title page template

Project title: LedgerLens — Oracle PL/SQL Transaction Monitoring and Audit System
Student name: `[Enter student name]`
Roll number: `[Enter roll number]`
Course: `[Enter course / MCA programme]`
Semester: `[Enter semester]`
Institution: `[Enter institution]`
Academic year: `[Enter academic year]`
Guide / instructor: `[Enter only if applicable]`

> Replace the bracketed fields with accurate personal and institution details. Do not add approvals, marks, or signatures that have not been given.

Abstract

LedgerLens is an educational bank-transfer processing and audit system implemented with Oracle SQL and PL/SQL. It models customers, accounts, transfers, review decisions, configuration and audit history. The `PKG_LEDGERLENS` package validates transfer requests, applies a configurable maximum amount, classifies requests at or above a high-value threshold, and either executes them or holds them for review. Account rows are locked in a consistent order before balances are updated, and a savepoint protects transfer processing from partial balance changes. SQL reports and an optional Flask dashboard make the lifecycle visible for demonstrations. Seed records are fictional. The design is intended for learning and is not a production banking or fraud-scoring system.

1. Introduction

Financial transaction systems need consistent balance updates, clear state transitions and records that help people investigate an event. This project demonstrates database programming techniques behind a small internal transfer workflow. Oracle owns the business rules; the optional web interface calls package procedures rather than changing balances itself.

2. Problem statement

A simple transfer example often omits concurrent account updates, auditability, review holds, configuration and failure rollback. LedgerLens provides one small, reproducible environment in which those database concepts can be discussed and demonstrated with deterministic fictional data.

3. Objectives

- Design related Oracle tables with keys, checks, timestamps and useful indexes.
- Implement reusable PL/SQL package routines for validation, transfer processing and review.
- Preserve balance integrity through row locks, revalidation and transaction savepoints.

- Demonstrate configurable threshold-based monitoring in complete and hold modes.
- Record lifecycle, status and configuration changes for audit queries.
- Provide reports, a repeatable PL/SQL test suite, beginner-oriented documentation and an optional Flask interface.

4. Scope and limitations

The system transfers funds between accounts within one Oracle schema. It supports synthetic accounts, configurable review threshold/mode/maximum, review decisions, audit history and reports. The dashboard is optional.

This project does not connect to a real bank, external payment rail, identity provider, sanctions screening service or fraud model. It has no authentication service, approval separation-of-duties workflow, production monitoring, backup plan, regulatory certification or formal security review. A high-value flag is not proof of fraud. The database test suite requires a running Oracle instance and must not be described as passed without execution.

5. Hardware and software requirements

- A computer able to run Oracle Database 19c+ or Oracle Database Free.
- SQL Developer or SQLcl and a dedicated learning schema.
- Git; GitHub is optional for hosting.
- Optional dashboard: Python 3.11+, Flask, `python-oracledb`, `python-dotenv`, and network access from the app to Oracle.

6. System architecture

The SQL client or Flask dashboard invokes `PKG_LEDGERLENS`. The package reads configuration, validates requests, locks account rows in account-ID order, and writes transfer, review and audit data. Triggers set account update timestamps and record generic transfer status changes. Reports query the normalized tables. The architecture document provides the request path and transaction ownership details.

7. Database schema and relationships

The core entities are CUSTOMERS, ACCOUNTS, TRANSFERS, AUDIT_LOG, SYSTEM_CONFIG and REVIEW_QUEUE. Supporting SYSTEM_OPERATORS, SYSTEM_REVIEWERS and CONFIG_AUDIT tables control and record privileged actions and threshold changes. A customer owns accounts; a transfer has a source and destination account; transfers have lifecycle audit events and review entries; configuration keys can have change-audit rows. The full Mermaid ER diagram is in ER_DIAGRAM.md.

Balances, transfer amounts and thresholds are decimal values. Oracle `NUMBER(18,2)` is used for money. Foreign keys protect references, unique constraints prevent duplicate emails/account numbers/references, and check constraints enforce valid account types, statuses, positive transfer amounts and nonnegative balances.

8. PL/SQL implementation

`PKG_LEDGERLENS` exposes:

- `CREATE_TRANSFER`: creates a request and returns its ID, reference, status, risk flag and message.

- `VALIDATE_TRANSFER`: checks amount precision/positivity, account existence and distinction, account states, balance and configuration limits.
- `PROCESS_TRANSFER`: locks the transfer and affected accounts, rechecks conditions and performs a debit/credit pair.
- `GET_HIGH_VALUE_THRESHOLD` and `SET_HIGH_VALUE_THRESHOLD`: read and authorizedly update the threshold, with configuration audit.
- `FLAG_FOR_REVIEW`: marks risk, creates at most one pending review and records the reason.
- `REVIEW_TRANSFER`: records approval/rejection; approval re-enters processing, rejection cannot execute.

The package does not commit. It uses savepoints to constrain its own rollback behavior. The caller commits or rolls back the wider transaction. `TRG_ACCOUNTS_UPDATED_AT` maintains a timestamp. `TRG_TRANSFERS_STATUS_AUDIT` writes a generic audit event on state changes. Package lifecycle events include creation, validation, execution, flagging, review and failures. These records supplement the status trigger rather than duplicating its purpose.

The package uses an explicit cursor for the two account locks and an ordered query so opposite-direction transfers acquire locks in the same account-ID order. A held transfer remains unexecuted until a reviewer approves it; approval triggers fresh validation while the account locks are held.

9. Threshold behavior

The seeded threshold is `50000.00`, and the rule is inclusive: `amount >= threshold` is flagged. `FLAG_AND_COMPLETE` is the default: a valid high-value transfer completes, then enters the review queue. `HOLD_FOR_REVIEW` marks the request as held and records a review before balances change. The configured `MAX_TRANSFER_AMOUNT` is a separate validation limit (`250000.00` in seed data). A review flag indicates that a person should inspect the transfer; it does not establish illicit activity.

10. Implementation and operation

Create a fresh schema, then run `database/01_schema.sql`, `02_seed_data.sql`, `03_packages.sql` and `04_triggers.sql` in that order. Run `05_reports.sql` for read-only views, `06_demo.sql` for two committed examples and `07_tests.sql` for rollback-isolated tests. The optional Flask UI obtains Oracle credentials from environment settings and calls package procedures through bind variables. Full steps appear in USER_GUIDE.md.

11. Sample input and expected output

Input	Expected state	Balance effect
`LL-ACC-1001` → `LL-ACC-1002`, `1000.00`	`COMPLETED`, risk `N`	Source -1000, destination +1000
Same accounts, `60000.00`, default mode	`COMPLETED`, risk `Y`, pending review	Source -60000, destination +60000
Same accounts, `60000.00`, hold mode	`HELD_FOR_REVIEW`, risk `Y`	No movement until approval
Inactive source or insufficient funds	`FAILED`	No movement

Input	Expected state	Balance effect
Review a held request as `REJECTED`	`REJECTED`	No movement

These are expected behaviors from the implementation; they are not represented as runtime observations in an Oracle environment unless the suite/demo has actually been run.

12. Testing methodology and actual results

The PL/SQL suite runs each case under a savepoint and rolls back its changes. It covers 17 checks, including validation, inclusive threshold equality, post-execution review disposition, audit, repeated execution and processor rollback after a forced destination-credit overflow. The final case is sequential repeated processing; an actual two-session concurrency check remains a separate follow-up.

Authoring-environment result: Python 3.13.14 compilation passed. A Flask smoke check rendered five GET pages and exercised three POST handlers with stubbed database calls; this verifies template/route behavior only, not Oracle integration. Required project files and local Markdown links resolved. Flask was available; `python-oracledb`, `python-dotenv`, SQLcl and SQL*Plus were absent, and no Oracle connection settings were configured. The schema/package was therefore not installed or compiled and `07_tests.sql` was not executed. Replace this record with actual Oracle output after running the project on a database; do not report expected results as observed results.

13. Conclusion

LedgerLens provides an understandable example of using PL/SQL packages, constraints, triggers, cursors, exception paths and audit tables to manage a transfer workflow. Its primary educational contribution is showing how validation, locks, state transitions and caller-owned transactions fit together. The database and dashboard should be verified on a real Oracle instance before any presentation claims runtime success.

14. Future enhancements

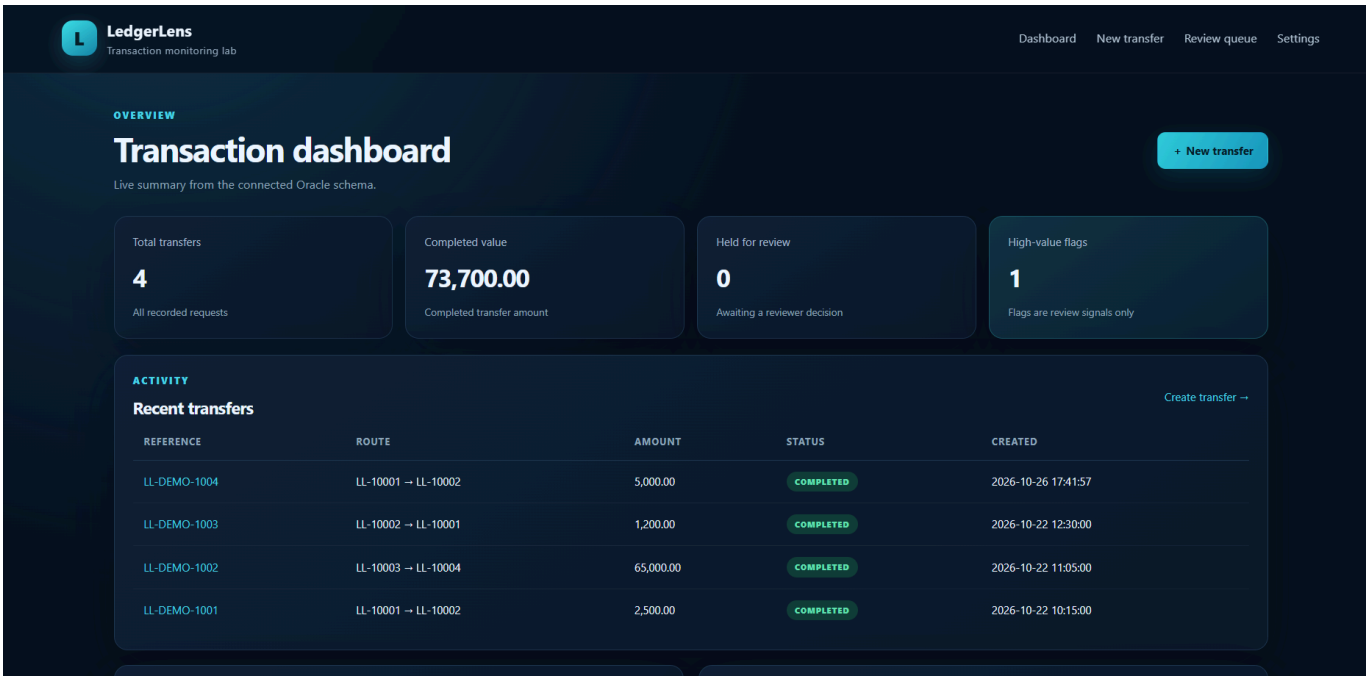
- Add authenticated users, role-based application access and independent reviewer identity.
- Add a transfer idempotency key and richer business rules.
- Add database migration/versioning, CI against Oracle Free, and multi-session concurrency tests.
- Add pagination, user-visible operational monitoring and accessibility review to the dashboard.
- Evaluate audit retention and tamper-evident controls for a production-grade system.

Screenshots and Results

This section presents screenshots of the LedgerLens web application demonstrating its main interface and features. The application was run in Demo Mode for interface and workflow demonstration.

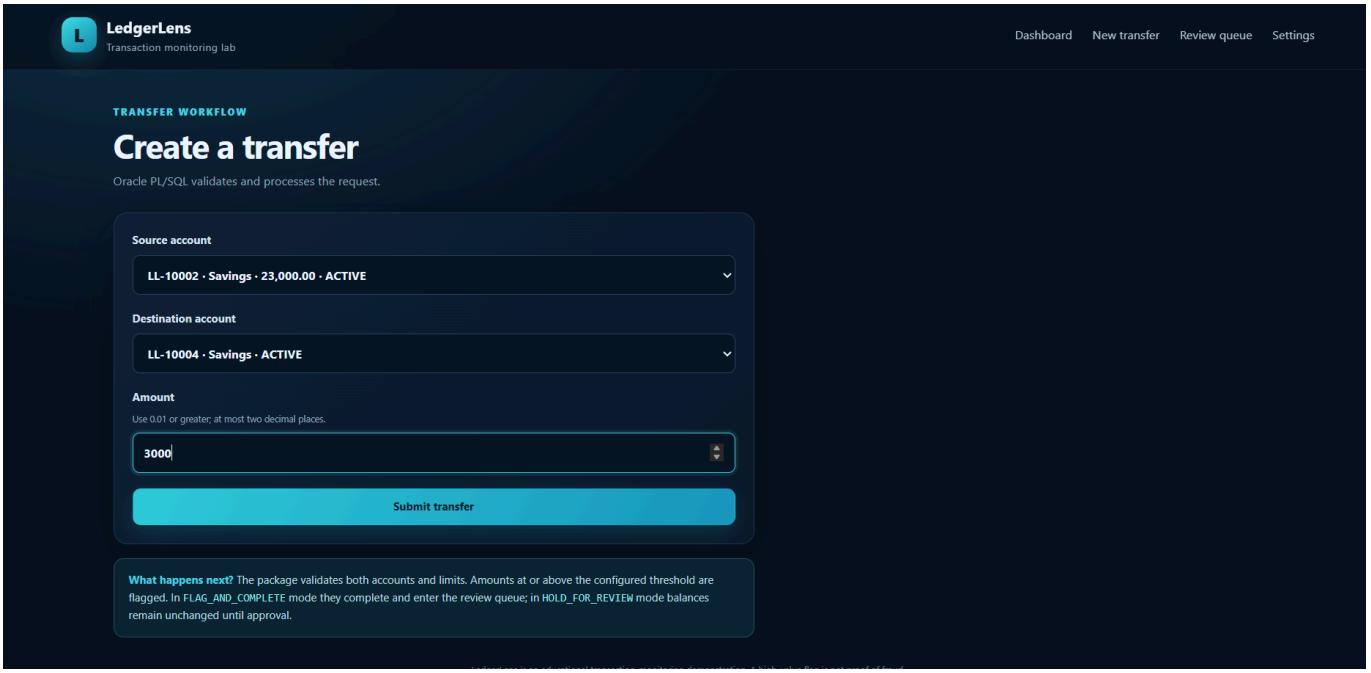
1. Dashboard

The Dashboard provides an overview of the LedgerLens application and its transaction monitoring features.



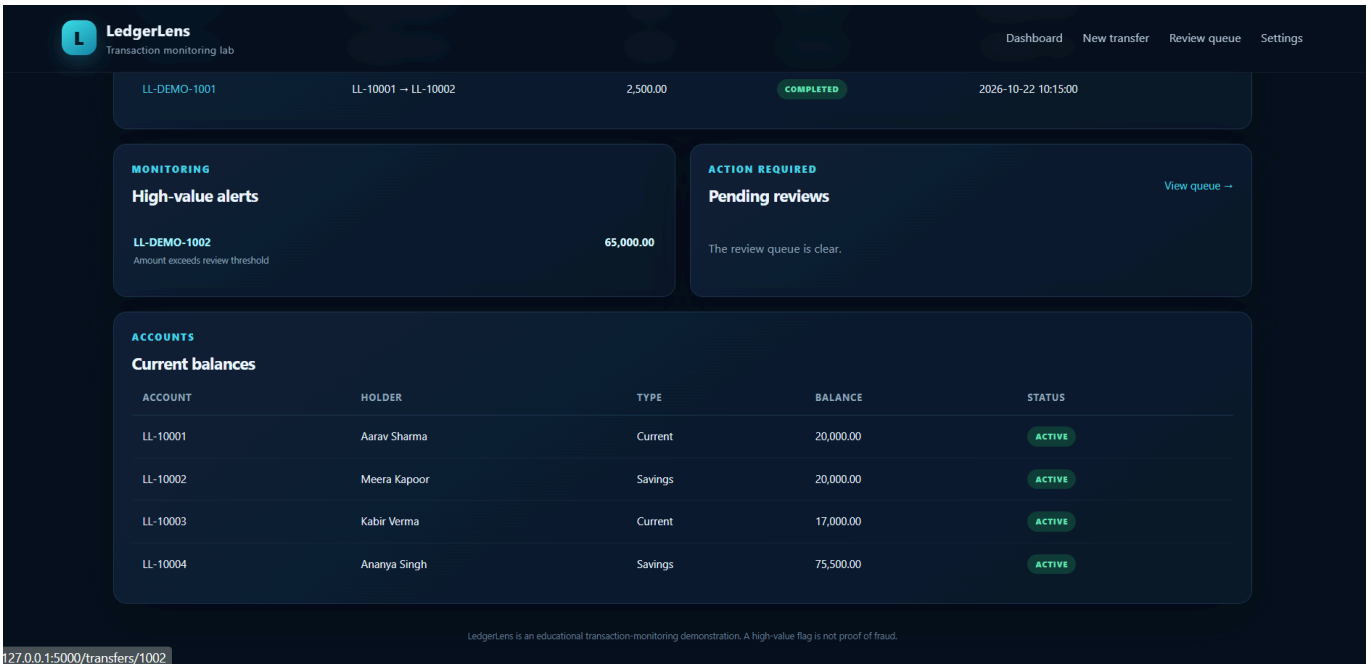
2. New Transfer

The New Transfer page allows users to enter transfer details and initiate the transaction workflow.



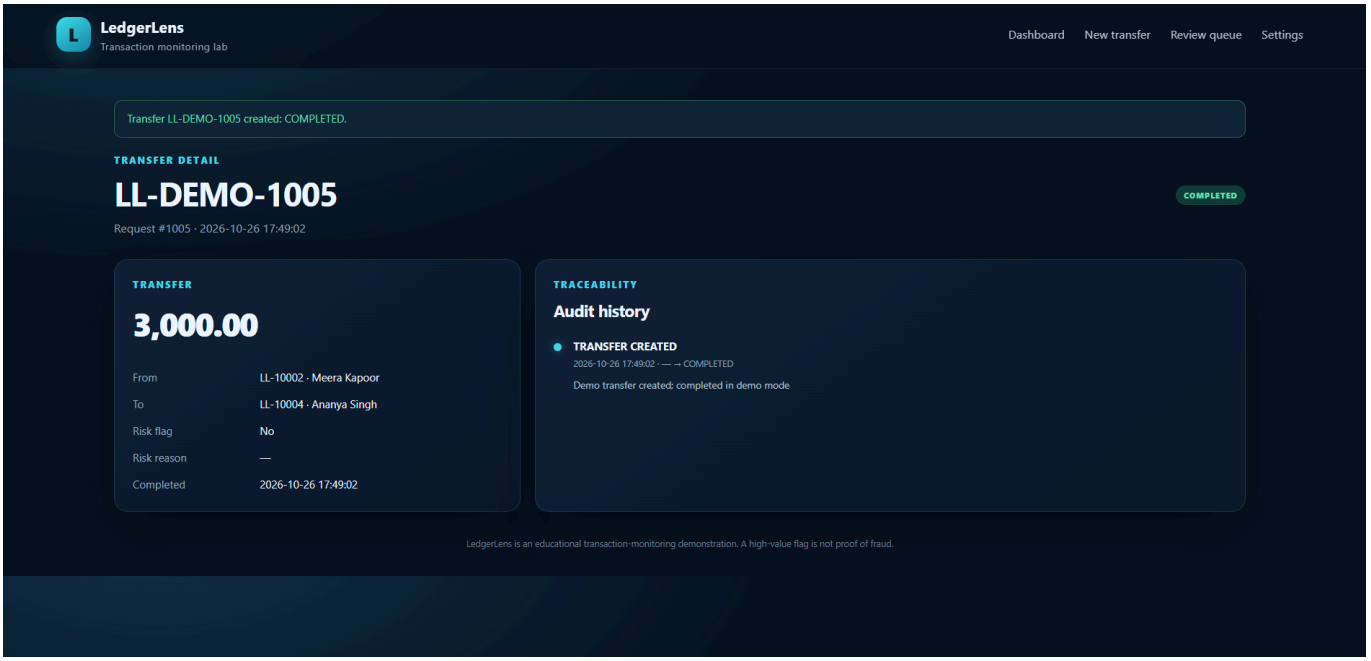
3. Reviews

The Reviews page provides an interface for reviewing transfers that require further attention.



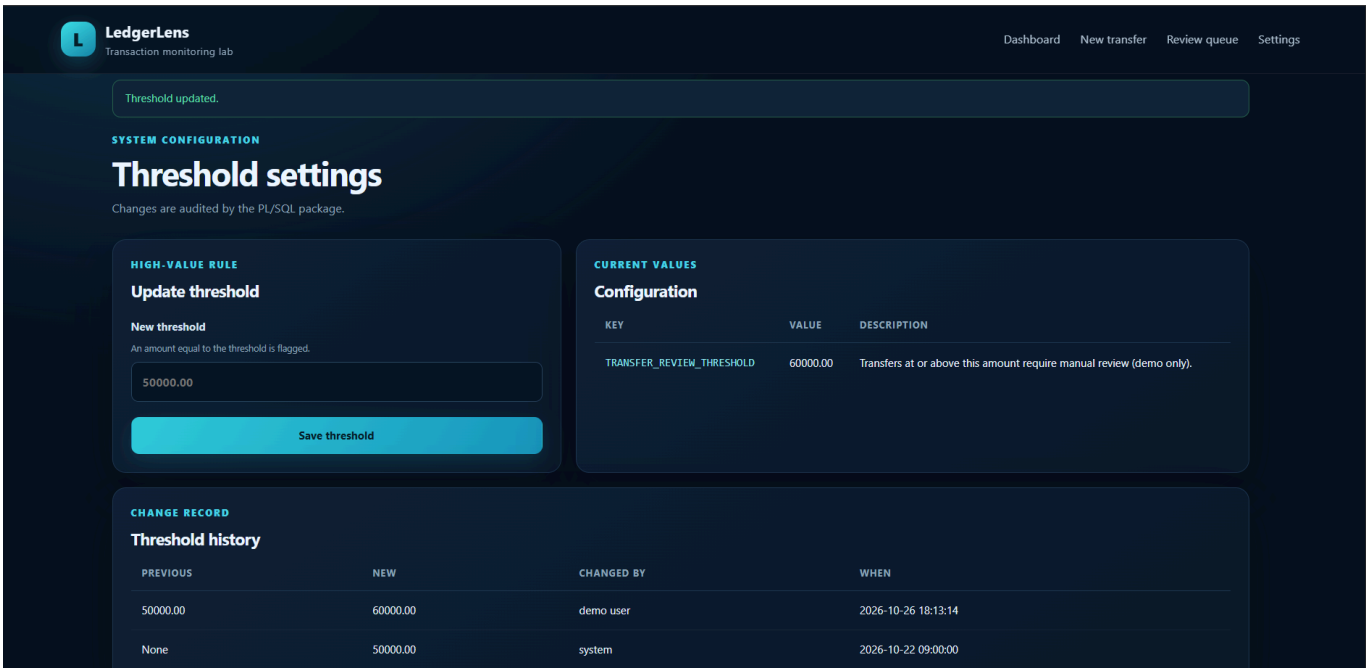
4. Successful Transfer

This screenshot demonstrates the successful transfer result displayed by the application during testing.



5. Threshold Settings

The Settings page allows the user to update the transaction monitoring threshold. The updated threshold was successfully saved during the demonstration.



Results Summary

The screenshots demonstrate the application's dashboard, transfer form, review interface, successful transfer result, and threshold configuration page. These screenshots document the demonstrated functionality of the Flask application in Demo Mode. Oracle Database connectivity was not verified as part of this demonstration.

References

- Oracle Database PL/SQL Language Reference: Oracle Database documentation.
- Oracle Database SQL Language Reference: Oracle Database documentation.
- Oracle Database Concepts, transactions and concurrency: Oracle Database documentation.
- Python documentation: Python 3 documentation.
- Flask documentation: Flask documentation.
- python-oracledb documentation: python-oracledb documentation.